

Sistemas Operativos

Práctica 3

Wenjie Chen - 100522255

Daniel Deblas Payo - 100522144

Xiya Ye - 100522159

5 de mayo del 2025



Índice

Descripción del Código.....	2
factory_manager.c.....	2
process_manager.c.....	4
queue.c.....	5
Batería de Pruebas.....	6
Conclusión.....	9

Descripción del Código

factory_manager.c

El programa en *factory_manager.c* es el programa principal, donde recibe como argumento un fichero que indica el número máximo de cintas, y los datos de cada cinta a crear, que se tratan del id de la cinta, el tamaño máximo de éste y el número de productos a generar. Para crear las cintas lo hace llamando a otra función, *process_manager*.

Para implementar este programa, lo primero que hacemos es importar las librerías que se van a necesitar. Además, importamos la función de *process_manager* que se encuentra en el archivo *process_manager.c*, esto lo hacemos con la función *extern*. También creamos la estructura de los parámetros que se pasará al hilo de *process_manager*.

Una vez hecho esto, podemos empezar con el *main*. Empezamos creando las variables que usaremos para la lectura del fichero, teniendo en cuenta que no hay un tamaño máximo para el fichero, entonces, hemos decidido usar memoria dinámica para poder reservar la memoria justa, por tanto, debemos usar la función *malloc* para reservar un espacio de memoria, y *realloc* cuando el buffer estuviera lleno. Esto lo hacemos tanto para *chars_leidos* como para *num_leidos* que será los buffers que almacenan los números en chars (que serán los que lee directamente del fichero) y los números pasados en enteros (que serán los que usemos). Al tener dos buffers, hemos de añadir también dos iteradores, *i* (para *chars_leidos*) y *pos* (para *num_leidos*).

Posteriormente, usamos un condicional para asegurarnos de que el número de argumentos sea el correcto, en caso contrario, debe saltar un error.

Para la lectura del archivo de entrada, lo primero que debemos hacer es abrirlo con la función *open* para lectura solo. Entonces, lo leemos con la función *read* de un caracter a un caracter almacenando dicho char en el buffer creado anteriormente. Para distinguir entre un número y otro, hemos puesto un condicional donde separe los números por un espacio. Además, siempre que *i* sea mayor que 0, significa que hay un número, por lo que añadimos el número al array, pero antes de hacerlo, nos tenemos que asegurar de que el buffer tiene espacio, por lo que lo comprobamos con un condicional. En el caso de que no tenga espacio, le multiplicamos al tamaño de buffer por dos, y usando dos arrays temporales para no tener problemas con las originales, le indicamos al los temporales el nuevo tamaño, y finalmente le asignamos al los arrays los nuevos. Hecho esto, podemos añadir el número a los buffers. Para esto, hemos decidido guardar el primer número (número máximo de cintas) en una variable aparte para que sea más fácil el acceso, por lo que cuando es el primero. Por tanto, le añadimos '10' al char y lo



convertimos en un entero con la función *atoi*, esto lo hacemos tanto para el primero como para el resto, siempre teniendo en cuenta que el número debe ser mayor que 0, y con la única diferencia de dónde almacenamos el valor. Una vez terminado el bucle, tenemos que tener en cuenta que como hemos leído todos los números con un espacio detrás, el último número puede no llevar un espacio, por lo que hacemos lo mismo para este último número.

Una vez leído el fichero de entrada, pasamos a crear los hilos. Primero creamos un array de tipo *sem_t* y usando *malloc* para reservar espacio, y como tenemos el máximo de cintas, reservamos el espacio pedido y esta vez, no va a ser necesario usar *realloc*. Ahora pasamos a inicializar los semáforos con *sem_init*, esto lo hacemos para todos, por lo que usamos un bucle *while*, para cuando $j*3$ sea menor que el número de enteros que haya en el buffer. Así, nos aseguramos de que inicialice para todos.

Para la creación de los hilos, usamos un array también, que sean de tipo *pthread_t*, también aprovechamos para crear un array que sean del tipo de la estructura que hemos creado al principio. Usando un bucle muy parecido al anterior, vamos dando valores a cada estructura y procedemos a crear el thread con *pthread_create*.

Para la sincronización de los hilos, usamos un bucle *for* para controlar los hilos creados. Primero, usando *sem_post*, despierte al hilo y espera a que termine con *pthread_join*. Como esperamos un resultado de los hilos, y los hilos solo devuelven un puntero, creamos un puntero y guardamos el resultado en esa dirección, y dependiendo del resultado se sigue ejecutando o o salta un error.

Finalmente, para que no haya problemas con la memoria, debemos liberar los recursos. Destruimos todos los semáforos con un bucle y la función *sem_destroy* y liberamos los espacios sobrantes que reservamos con *malloc* anteriormente. Y devolvemos 0.

process_manager.c

El archivo *process_manager.c* crea pares de producer-consumer, entre cada producer-consumer hay una cola en el cual producer mete elementos y consumer los coge, para la creación y manipulación de la cola usamos las funciones del archivo *queue.c*.

Primero de todo hemos creado dos estructuras: uno para pasar datos a los hilos (productor y consumidor), como: el ID del hilo, número de elementos a producir/consumir y puntero a la cola compartida; y otro para inicializar la función *process_manager*, como: ID del *process_manager*, tamaño máximo de la cola, número de elementos que se deben producir.

Luego creamos una función *producer* que se encarga de crear elementos y ponerlos dentro de la cola compartida. Recibe una estructura de datos como el ID, la cantidad de elementos a producir y un puntero a la cola. Hacemos un bucle en el cual vamos creando los elementos asignándole un número de edición (índice del elemento), el ID y ver si es el último elemento producido. Con *queue_put* insertamos los elementos en la cola, en caso de error se imprime el mensaje indicado y si no se finaliza correctamente.

Después creamos la función *consumer* que coge los elementos del hilo pasados por el producer. Recibe una estructura de datos como el ID, cantidad de elementos a consumir y la referencia a la cola. Usamos un bucle en el cual vamos extrayendo cada vez un elemento de la cola usando *queue_get*, en caso de que hubo error al obtener un elemento imprimimos el mensaje de error, en otro caso liberamos memoria del elemento extraído y finalizamos la función.

Finalmente creamos una función principal *process_manager* que se encarga de coordinar el proceso de producer, consumer y las colas. Recibe como argumentos: el ID, tamaño máximo de la cola y la cantidad total de elementos que se deben producir/consumir. Primero vemos si los argumentos son válidos, en caso de fallo imprimimos un mensaje y sino imprimimos otro. Luego inicializamos una cola con el tamaño especificado, en caso de fallo termina con error. Después reservamos memoria para las estructuras de datos, uno para producer y otro para consumer y creamos los hilos. Si alguno de ellos no se puede crear limpiamos todo y finalizamos con error. Una vez ambos hilos funcionen, espera a que terminen y liberamos la memoria usada para los hilos y comprobamos si la cola está vacía. Si aún hay elementos o si la destrucción de la cola falla, mandamos un error. Si todo sale bien, imprimimos un mensaje de éxito indicando que el proceso terminó correctamente y se cierra sin errores.

queue.c

El programa *queue.c* implementa una cola circular sincronizada. Esta cola permite insertar y extraer elementos entre diferentes hilos productores y consumidores. Para que la cola funcione correctamente, esta sigue una estructura que está definida en *queue.h*. En primer lugar, esta estructura contiene un buffer dinámico que almacena los elementos. En segundo lugar cuenta con cuatro variables de control (*tamano*, *num_elementos*, *elem_leer*, *elem_escribir*). En tercer y último lugar, esta estructura cuenta con tres elementos de sincronización (*mutex*, *not_empty*, *not_full*). Nuestro programa cuenta con diferentes funciones.

La primera de ellas es *queue_init*. Esta función sirve para inicializar la cola. Lo primero que hace es comprobar que el tamaño que recibe es correcto. Después, reserva memoria en función del tamaño. Por último, se procede a inicializar las variables de control y de los mecanismos de sincronización.

La segunda función es *queue_put* y es la que nos permite añadir un nuevo elemento a la cola. Lo primero que hace este método es comprobar que el elemento a insertar sea correcto. En caso de que sea correcto se procede a la inserción. Cuando se va a insertar el elemento, se bloquea el mutex para controlar el acceso a la cola. En caso de que la cola esté llena, se espera hasta que se pueda insertar el elemento. Posteriormente, para añadir el elemento, se copia el contenido de **x* en la posición indicada por *elem_escribir* en el buffer. Por último se actualiza el valor de *elem_escribir* y *num_elementos*.

La tercera función es *queue_get* y es la que se dedica a extraer un elemento de la cola. Lo primero que hace es proteger el acceso a la cola bloqueando el mutex. Después, comprueba si la cola está vacía y espera si es cierto. Para extraer el elemento, la función usa *malloc* para crear una copia del elemento a devolver. Después se copia este elemento en la variable *elem_leer* y se actualizan el resto de variables. Por último se notifica que la cola no está llena, se desbloquea el mutex y se devuelve el elemento.

La cuarta y quinta función están relacionadas entre sí. Son las que controlan si la cola está llena o vacía. Para ello, ambas bloquean el mutex, comprueban si está llena o vacía, desbloquean el mutex y devuelve el resultado.

La última función es la que destruye la cola. Esta función, libera la memoria del buffer, restaura las variables de control y destruye el mutex y las condiciones para liberar los recursos asociados a los hilos.

Batería de Pruebas

Fichero de entrada	Descripción de la prueba	Resultado esperado	Resultado obtenido
fichero que no existe	Comprobamos que no funciona cuando el fichero no existe.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.
sólo ./factory	Se comprueba que salta error cuando no recibe los argumentos suficientes.	ERROR: [ERROR][factory_manager] Process_manager with id 0 has finished with errors.	ERROR: [ERROR][factory_manager] Process_manager with id 0 has finished with errors.
0	Se comprueba el funcionamiento cuando se quiere crear 0 cintas.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.
0 1 2 3 2 1 3	Asimismo, también comprobamos que funcione para 0 cintas, pero con los datos de las cintas.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.
-2 1 2 3	Comprobamos el funcionamiento para cuando el número máximo de cintas es negativo.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.
1 3 5 0	Comprobamos que salta error cuando el número de elementos a producir es 0.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.

Fichero de entrada	Descripción de la prueba	Resultado esperado	Resultado obtenido
13 0 3	Se comprueba que cuando el tamaño de una cinta es 0 salta error.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.
13 -6 3	Se comprueba que salta error cuando el tamaño de cinta pedido es negativo.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.
13 5 2	Se comprueba que pidiendo en una cinta producir un número de productos menor al tamaño de la cinta funciona correctamente.	Ejecución correcta	Ejecución correcta
13 5 5	Se comprueba que funciona cuando se le pide justo el mismo número de productos que el tamaño de la cinta	Ejecución correcta	Ejecución correcta
13 5 10	Comprobamos que funciona cuando el número de productos a producir es mayor que el tamaño de la cinta.	Ejecución correcta	Ejecución correcta
3	Se comprueba el funcionamiento cuando no pedimos que cree ninguna cinta, pero sí hay un máximo de cintas.	Ejecución correcta	Ejecución correcta

Fichero de entrada	Descripción de la prueba	Resultado esperado	Resultado obtenido
3 1 2 3 4 3 1 2 3 4	Comprobamos que para un número pequeño de máximo de cintas como 3, pidiendo que cree justo 3 cintas.	Ejecución correcta	Ejecución correcta
3 1 1 3 2 1 4	Con un número pequeño n (3) para el valor de max cintas comprobamos que funcione con n - 1 (2) cintas.	Ejecución correcta	Ejecución correcta
3 1 1 3 2 1 4 3 4 1 4 5 2	Con un número pequeño n (3) para el valor de max cintas comprobamos que funcione con n + 1 (4) cintas.	ERROR: [ERROR][factory_manager] Invalid file.	ERROR: [ERROR][factory_manager] Invalid file.
10 1 2 3 2 3 4 3 4 5 4 5 6 5 6 7 6 7 8 7 8 9 8 9 10 9 10 11 10 11 12	Comprobamos que para un número grande de máximo de cintas como 10, pidiendo que cree justo 10 cintas.	Ejecución correcta	Ejecución correcta
10 1 2 3 2 3 4 3 4 5 4 5 6 5 6 7 6 7 8 7 8 9 8 9 10 9 10 11	Con un número grande n (10) para el valor de max cintas comprobamos que funcione con n - 1 (9) cintas.	Ejecución correcta	Ejecución correcta

Conclusión

Gracias a la realización de esta práctica hemos podido mejorar nuestros conocimientos sobre la creación y gestión de múltiples hilos usando la librería POSIX Threads. Por otro lado, también hemos podido aprender a sincronizar el acceso a los recursos por parte de diferentes procesos.

Al crear el programa *queue.c* nos hemos encontrado con diferentes problemas. La primera dificultad que nos hemos encontrado ha sido controlar correctamente la sincronización de la cola y gestionar la memoria dinámica de la misma. Sin embargo, la mayor complicación que hemos tenido ha sido diseñar la cola para que pueda ser utilizada por diferentes productores y/o consumidores al mismo tiempo.

Por otro lado al realizar el programa *process_manager.c*, al estar entre el *factory_manager* y el *queue*, tenemos que tener mucho cuidado a la hora de utilizar las colas que sea compatible con lo escrito en *queue.c* y las entradas recibidas sean iguales que las salidas que mandan el *factory_manager*. Hemos tenido un par de problemas como: el acceso de los datos tanto producir y consumer a una misma zona de memoria reservada, que se compartía una cola para todos y liberar la memoria cuando sea necesario.

Por último, al crear el programa *factory_manager.c* hemos tenido algunas dificultades a la hora de trabajar con una memoria dinámica, ya que cuando no se libera bien la memoria, puede haber problemas a la hora de compilar o al ejecutar el programa. Otro problema que hemos tenido ha sido que en principio, nuestro programa no leía el último número del fichero de entrada cuando éste no terminaba por un espacio. Para resolverlo hemos usado un iterador que indicara cuándo hay número y cuándo no (para ello, inicializamos el iterador cada vez que leemos el número). Aparte de esto, nuestro gran problema ha sido la implementación de los semáforos para la sincronización entre hilos. Ya que nuestra idea principal ha sido usar un único bucle, y en él hacer todo lo relacionado con los hilos (crearlo, esperar a que termine y liberar los recursos), pero no tenía sentido, ya que no sería necesario el uso de los semáforos. Así que al final, nos decantamos por usar un bucle para cada acción.

En general, esta práctica nos ha servido para entender mejor el uso de los semáforos para la sincronización entre procesos, y también cómo trabajan los programas sobre productor-consumidor.